

webRagSys — 社内文書検索・AIチャットシステム

社内文書を登録・管理し、ユーザーからの質問に対して登録文書を検索したうえで生成AIが回答するWeb APIシステム。

システム概要

- 文書管理（CRUD）とAIチャット（RAG）を提供するFastAPIシステム
- 処理の本質：Python → API → DB → LLM → RAG
- 登録文書をテキスト分割→Embedding→Vector Databaseへ保存し、質問時に類似検索してLLMへ渡す

主な機能

- 文書登録、一覧取得、詳細取得、更新、削除
- AIチャット：質問→Embedding→類似文書検索→LLM回答
- RAG：LangChain利用（TextSplitter、Embeddings、VectorStore、Retriever、Chain）
- 既定は課金回避の偽実装（FakeEmbeddings + Fake LLM）。本番のみ外部LLM API使用

使用技術

項目	内容
言語	Python 3.10+（実測3.11）
API	FastAPI、Pydantic
DB	PostgreSQL + pgvector、SQLAlchemy
RAG	LangChain
LLM	外部LLM API（モデル非依存）、既定は偽実装
テスト	pytest
Lint	ruff
環境	Docker／docker-compose、Git／GitHub

システム構成

```
webRagSys/
├── AGENTS.md
├── README.md
├── config.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .env.example
├── src/api/          → main.py、routers/documents.py、routers/chat.py
├── src/schemas/     → Pydanticスキーマ
├── src/db/           → database.py、models.py
├── src/rag/          → chunker.py、embeddings.py、vectorstore.py、chain.py
├── src/llm/          → provider.py、fake_provider.py
```

```
|—— src/utils/  
|—— tests/  
|—— docs/          → beginner_overview.md、handover.md  
|—— diagrams/       → Mermaid図（.mmd+.md+.pdf）
```

セットアップ方法

```
# 1. 依存導入  
pip install -r requirements.txt  
  
# 2. 環境変数ファイル作成（公開用見本から複製）  
copy .env.example .env  
  
# 3. .env編集（偽実装のままならキー不要）  
# USE_FAKE=true  
# 本番のみ以下を設定  
# OPENAI_API_KEY=xxxx  
# USE_FAKE=false  
# DATABASE_URL=postgresql://user:password@db:5432/webraq
```

起動方法

```
# ローカル起動  
uvicorn src.api.main:app --reload  
  
# Docker起動（予定）  
docker compose up --build
```

API一覧

メソッド	エンドポイント	内容
GET	/documents	文書一覧取得
GET	/documents/{id}	文書詳細取得
POST	/documents	文書登録
PUT	/documents/{id}	文書更新
DELETE	/documents/{id}	文書削除
POST	/chat	AIチャット

リクエスト／レスポンスはJSONを使用する。

RAGの処理概要

文書
↓
テキスト分割 (LangChain TextSplitter)
↓
Embedding (本番：外部API／既定：FakeEmbeddings)
↓
Vector Databaseへ保存 (pgvector)

ユーザー質問
↓
Embedding
↓
類似文書検索 (pgvector+LangChain Retriever)
↓
検索結果をコンテキスト化
↓
LLM (本番：外部API／既定：Fake LLM)
↓
回答 (+参照文書ID)

使用方法

```
# 文書登録例
curl -X POST http://localhost:8000/documents ^
-H "Content-Type: application/json" ^
-d "{\"title¥\": ¥\"休暇規定¥\", ¥\"content¥\": ¥\"年次休暇は...¥\"}"

# 質問例
curl -X POST http://localhost:8000/chat ^
-H "Content-Type: application/json" ^
-d "{\"question¥\": ¥\"休暇申請の手順は?¥\"}"
```

テスト方法

```
# 全テスト（偽実装のため外部通信なし）
python -m pytest tests -q

# Lint
python -m ruff check src tests config.py
```

テスト内容（予定）：

- 文書：登録、取得、更新、削除、存在しないIDのエラー
- チャット：質問送信、偽LLM回答、登録文書利用、APIエラー時処理

ドキュメント

- docs/beginner_overview.md：Python中級・RAG初心者向け解説（テスト後に作成）

- docs/handover.md : 進捗管理 (方針→実装→結果→残タスク→実行コマンド)
- diagrams/ : フローチャート、シーケンス、クラス、mindmap、状態遷移 (テスト後に作成、PDF付き)

今後の拡張案

- ユーザー認証、文書ファイルアップロード、PDF／Word対応
- 文書カテゴリ管理、会話履歴、回答の出典表示
- Webフロントエンド、クラウドデプロイ、CI/CD
- 高度なRAG検索、AIエージェント機能

完成条件（初期版）

1. FastAPI起動、PostgreSQL接続、文書CRUD動作
2. LLM API利用可（本番）／偽実装動作（既定）
3. Embedding化、Vector保存、類似検索、RAG回答
4. Docker構築、GitHubでREADME確認、基本テスト実行

注意点

- .env はGit管理対象外。APIキー等を公開しないこと
- docsとdiagramsはテストの後に作成し、PDF生成すること